

OOPJ Day 1 Fundamentals

Mon, 07 Sept 26

Course Structure and Expectations

- 15-day Java fundamentals module, the foundation for everything that follows
 - Followed by DSA (10 days), then web programming in Java (12 days)
 - Final 3 days of web programming are the direct basis for final projects and interviews
 - Entire preceding curriculum (Java, DSA, web programming) is preparation for those 3 days
- Missing even one session creates compounding gaps
 - Java concepts build sequentially; one missed session makes the next near-impossible
 - Described as “near to impossible” to recover without attending every class
- Session hours: 9 AM to 1 PM daily, with two 10-minute breaks
 - First break around 10:25 AM, second around 11:40 AM (per CDAC norms)
 - Online delivery; if internet drops, instructor reconnects via phone
- Assignments and exercises uploaded to Git before 8:45 AM on the day of each session
 - First 15–20 minutes of each session used for revision and doubt-solving via screen share
 - Students share their screen and work through problems live in front of the group
 - Most problems are common across students, so group debugging benefits everyone
- No notes distributed by the instructor; students must take their own during the session
 - Specific diagrams and screenshots the instructor considers important will be shared separately
 - Source code shared as images only so students are forced to type it, not copy-paste
 - Typing reinforces understanding; copy-pasting does not
- Expected self-study: minimum 8 hours after each session
 - First 1–2 days may only require 2–3 hours, but workload increases quickly
 - Assignments and exercises drive the bulk of that time
 - Complete that day’s work on the same day; do not carry pending work into the next session
- Attendance with camera on is mandatory; missing a session is strongly discouraged
 - “If you are not feeling well, please make sure you’re sitting with your cameras on”

Coding Standards and Naming Conventions

- Industry is very particular about how variables are named; this directly affects interview performance
 - Common bad habit: using i, j, k as variable names because textbooks used them
 - Authors used i because it was personally relatable (first-person “I”), not because it was correct
 - Students then learned j for a second variable, k for a third, skipping A–H entirely
 - “When you went to school, didn’t you learn from ABCD rather than IJK?”
 - Acceptable alternatives if you want to convey iteration:
 - iTemp, iVal, iNum: the prefix i signals integer type, the suffix names the purpose
 - This convention makes the variable’s type and role immediately readable
 - Class naming: avoid “my” prefixes (MyClass, MyCar, MyStudent)
 - A real-world object like Mukul is an instance of Man, not “my man”
 - “My” prefix carries no semantic value and is considered poor practice
 - When logic is unclear or variable purpose is non-obvious, add inline comments
 - Example: `// integer variable for holding a numerical value` next to `int num1 = 0;`
 - Documentation is the formal alternative; comments are the practical substitute during development
 - This matters when someone else (or your future self) maintains the code after you leave a project
 - Real-world scenario: you join Infosys, work on a project for 2 years, then move to Zensab
 - The person replacing you must understand your code
 - Randomly named variables (a, b, c, i, j) make that impossible without documentation
 - Good naming conventions reduce handover friction and prevent misinterpretation
-

Reusability and Libraries

- Core problem: the same data structure (e.g., linked list) gets rewritten 20–30 times across projects
 - What actually changes each time? Only the data type being stored
 - The structure (start, end, current, node), methods (add, delete, get first, get next, get last), and logic remain identical
 - Rewriting the same code repeatedly wastes man-hours and reintroduces the same bugs every time
- Solution: encapsulate into a reusable library, written once and shared across applications
 - Two advantages: bugs are fixed once in one place (robust solution), and development time is saved on every subsequent use
 - Example: SAP ERP billing module is generic enough to serve a shopkeeper, Amazon, or any online store

- Any application needing billing just attaches the module and customizes as needed
 - APIs (Application Programming Interfaces): the formal name for these reusable library functions
 - Full form read in reverse: an Interface, used by Programmers, for developing Applications
 - printf, scanf, fopen, fwrite, fclose are all APIs from the C standard library
 - printf abstracts away the hardware-level pixel-setting logic; you just call it
 - Roles defined by API usage:
 - Developer: the person who writes and publishes the API (e.g., Prajwal writes a linked list library)
 - Programmer: the person who uses someone else's API to build their application (e.g., Sahil uses Prajwal's library)
 - In practice, students play all three roles: user, programmer, and developer
 - Frameworks vs. libraries:
 - Libraries: fully pre-built, non-modifiable; you use them exactly as provided (e.g., printf)
 - Frameworks: partially modifiable via callbacks, overrides, or extension points (e.g., Spring, Hibernate)
 - Frameworks have a fixed skeleton you cannot change, plus selective customization hooks
 - Primary difference: libraries offer reuse; frameworks offer reuse plus controlled customization
-

Programming Language Fundamentals

- Computers originally operated only in low-level (binary) language: 0 = off, 1 = on
 1. Writing programs in binary is possible but extremely cumbersome, lengthy, and tedious
 2. As programming matured, the goal became making life easier for programmers, not the other way around
- High-level languages evolved to be human-readable (primarily English-like syntax)
 1. Language A → Language B → Language C, developed sequentially, not randomly
 2. C has been evolving for 70–75 years and is still being developed today
 3. ProC (Professional C) is an extension used in embedded systems
 4. Windows and Linux operating systems are primarily built on C
- The language creator defines two foundational components:
 1. Syntax: the structural rules for how every statement must be written
 - Where semicolons go, how functions open and close with braces, how arguments are passed with parentheses
 - Example: `int item;` is syntactically valid because it ends with a semicolon

2. Semantics/Grammar: the ordering and logical rules for how elements must be arranged
 - Data type must precede variable name: `int item` is correct; `item int` is grammatically wrong
 - Return type must precede function name: `void display()` is correct; `display() void` is wrong
 - A statement can be syntactically valid but grammatically incorrect simultaneously
 - Analogy: RTO (language creator) defines traffic rules; the traffic officer (compiler) enforces them
 1. The driver (programmer) is the one who may break the rules
 2. The officer does not define the rules; he checks whether the driver follows them
 - Compiler's two primary roles:
 1. Checks that the programmer's code follows the syntax and grammar defined by the language creator
 2. Converts valid high-level code into machine-dependent binary code (.obj file)
 - Compiler stages (simplified from the full 7–8 internal stages):
 1. Preprocessing: resolves all directives starting with #
 - `#include <stdio.h>` is replaced inline with the full contents of `stdio.h` (~70,000 lines)
 - `#define MAX 90` replaces every occurrence of `MAX` with `90` throughout the file
 - No syntax checking happens here; it is purely textual substitution
 2. Syntax and grammar checking: validates all statements against language rules
 3. Optimization: balances performance vs. memory utilization based on compiler settings
 - High-performance mode: uses more memory for faster execution
 - Memory-conservative mode: reduces memory usage at the cost of performance
 4. Code generation: converts the validated, optimized code into machine-dependent binary (.obj)
 - Inline function replacement happens during compilation proper, not during preprocessing
-

Build Toolchain: Compiler, Linker, Debugger

- Five prerequisites for any programming language development environment:
 - Text editor: write source files (.c, .cpp, .h); any plain-text editor works (Notepad, vi, VS Code)
 - File extension (.c, .cpp) is a convention for identifying the language used, not a technical requirement
 - .h files (header files) are also created via a text editor
 - Compiler: checks syntax/grammar and converts source to binary object file (.obj)

- Reads the source file character by character against the language rules
 - Only checks that function calls match their prototypes; does not verify that definitions exist
 - That is the linker's job, not the compiler's
- Linker/Assembler: takes the .obj file and produces the final executable
 - Attaches all required library files (.lib) to the binary
 - Maps every function call to its corresponding definition (internal linking)
 - Adds the COF (Common Object File Format) header and footer so Windows can execute the file
 - Linux uses a different binary format; the concept is the same
 - Analogy: the .obj file is like a BMW engine; it is functional but incomplete without the rest of the car
- Libraries: pre-compiled binary files containing the actual definitions of declared functions
 - stdio.h declares printf; stdio.lib contains printf's compiled binary definition
 - math.h → math.lib; string.h → string.lib, and so on
 - Business logic is never exposed in header files; it is always shipped as compiled .lib binaries
 - Even if you open a .lib file, it is machine code and unreadable to humans
- Debugger: a tool for identifying and fixing logical errors (bugs) after the executable is built
 - Not responsible for syntax or grammar errors; those are the compiler's domain
 - Works by allowing the programmer to set breakpoints: lines where execution pauses
 - From a breakpoint, the programmer steps through code one statement at a time
 - Watch window shows live variable values (e.g., the current value of num, the address held by a pointer)
 - Example bug: while(num < 90); with a trailing semicolon creates an infinite loop; hello world never prints
 - This is a logical error, not a syntax error; the compiler accepts it, the debugger helps find it
- Header files (.h) contain only function prototypes (declarations), never definitions
 - Confirmed by opening stdio.h in Visual Studio: line 413 shows int printf(const char*, ...) — a declaration only
 - One header file can include other header files (e.g., stdio.h includes other system headers at line 56)
 - When #include <stdio.h> is preprocessed, the entire contents of stdio.h are dumped into your source file
- The linker is the entity that enforces the main function as the entry point
 - It places the starting memory address in the COF header of the .exe
 - The OS reads that address and begins execution there; it does not understand function names, only addresses

- Changing the entry point requires an explicit instruction to the linker; main is simply the default
-

The main Function

- main is the linker's default entry point when building the executable
 1. With 20 functions in a .c file, the linker has no way to know which to call first without a convention
 2. The rule: the linker always looks for main as the starting function unless explicitly told otherwise
 3. Renaming main to starter or fn2 causes a linker error: undefined reference to main (or WinMain on Windows)
 4. Compiler compiles the renamed function without error; the linker fails because it cannot find main
 5. Entry point can be customized by instructing the linker to use a different function's memory address
- Return type of main:
 1. void main: the function returns nothing to whoever launched it
 2. int main: returns a numeric status/error code to the calling process
 3. Return codes have two primary uses:
 - Batch file / parent process logic: if app A returns 5, launch app E; if it returns 200, launch app B
 - Windows Event Viewer (Application Log): return codes are logged so administrators can diagnose issues
 - Security log: tracks logon/logoff events, failed password attempts, timestamps
 - Application log: records warnings, errors, and success events per application
 4. return 0 simply returns the integer value 0; it does not inherently mean "success" — that meaning is defined by the calling context
- Command-line arguments (int argc, char* argv[]):
 1. Allow values to be passed into the program at launch time from the command line or a batch file
 2. argc: count of arguments; argv: array of argument strings
 3. Example: notepad F:\customping.bat — "F:\customping.bat" is passed as argv[1] to Notepad's main
 4. Notepad checks whether an argument was received; if yes, it opens that file
 5. copy source destination: "copy" is the program, source and destination are two command-line arguments
 6. winword.exe filename /p: opens Word's print dialog directly without opening the UI

7. Compiler and linker are not involved at runtime; they no longer exist once the .exe is built
 - Opening and closing curly braces {} serve two purposes:
 1. Define the scope of the function: all statements between them belong to that function
 2. Trigger stack allocation and deallocation: memory for local variables is allocated at { and destroyed at }
 - Variable declarations are processed at the opening brace; the debugger skips over them when stepping
 - This is why local variables are called auto variables: memory is automatically managed
-

Scope, Lifetime, and Memory Segments

- Scope and lifetime are two distinct concepts, often confused:
 - Scope: the region of code where a variable is accessible (can be read or written)
 - Lifetime: the duration for which a variable occupies memory (exists on the stack or heap)
 - A variable can be alive in memory but out of scope (inaccessible by name)
- Worked example with num1 (in main) and data (in display):
 - num1 is declared in main; its scope and lifetime span the entire main function
 - data is declared in display; its scope and lifetime are limited to the display function
 - When display is called from within main, num1 is still alive (main has not returned)
 - num1 is out of scope inside display (cannot be accessed by name), but its memory at address 200 still exists
 - data is destroyed the moment display's closing } executes; it is both out of scope and out of lifetime
 - Attempting to access data after display returns yields garbage or a dangling pointer situation
- If display is never called, data is never allocated; its scope and lifetime are both zero
- Global variables:
 - Declared outside all functions; scope covers the entire .c file where they are defined
 - Lifetime spans the entire program execution
 - To use a global variable defined in main.c from within function.c, declare it with extern int global;
 - extern tells the compiler: "this variable exists somewhere; the linker will resolve it"
 - If the variable was never actually defined anywhere, the linker throws: unresolved external symbol global
 - Java has no equivalent of extern; this is C-specific

- Three logical memory segments (classifications for understanding, not physical partitions in RAM):
 - Stack (local store): local variables; auto-allocated at { and deallocated at }; LIFO order
 - Heap (free store): dynamically allocated memory via malloc, calloc, new; manually or GC-managed
 - Global/Static store: global and static variables; persist for the full program lifetime
 - RAM itself is just memory; the OS allocates and recycles addresses continuously across programs
 - When a program terminates, the OS does not zero out its RAM; it simply revokes access to those addresses
 - The data physically remains until overwritten by another program or the system restarts (RAM is volatile)
 - Cyber forensic tools can map live RAM and recover open files, passwords, and other data from terminated processes
 - OS avoids zeroing memory on termination deliberately to avoid wasting performance on cleanup
 - Perfect definition of lifetime (student-contributed, confirmed by instructor):
 - “The life of a variable is the time for which memory stays allocated to it”
-

Pointers and Dangling Pointers

- Pointer: a variable that holds the memory address of another variable
 - Standard definition: “a pointer is a variable that holds the memory address of another variable”
 - A pointer is itself a variable; it occupies memory just like any other variable
- Size of a pointer is determined by the address bus width, not the data type it points to:
 - 16-bit architecture: 2 bytes; 32-bit architecture: 4 bytes; 64-bit architecture: 8 bytes (if full 64-bit addressing)
 - int*, float*, long*, char*, double*, struct* — all are 4 bytes on a 32-bit system
 - The pointer only needs to store a memory address; address size is fixed by the architecture
 - Interview tip: state the answer confidently; “I think it might be...” signals uncertainty even if the answer is correct
- Data type sizes on 32-bit architecture (for reference):
 - int: 4 bytes; float: 4 bytes; long: 4 bytes; double: 8 bytes; char: 1 byte
 - struct with int + int + double: 4 + 4 + 8 = 16 bytes
- Pass by value vs. pass by reference:
 - Pass by value: a copy of the variable’s value is made in the called function
 - Changes inside the called function affect only the copy; the original is unchanged

- Example: passing num1 to display creates a new variable ptrNum1 at a different address holding the same value
 - Modifying ptrNum1 = 88 changes address 300, not address 200 where num1 lives
 - When display returns, ptrNum1 is destroyed; num1 still holds 0
- Pass by reference (via pointer): the memory address of the original variable is passed
 - The called function receives the address and can directly modify the original
 - Use & (address-of operator) to pass the address: display(&num1)
 - Declare the receiving parameter as int* ptrNum1 to accept a pointer
 - Use * (dereference operator) to follow the pointer and modify the original:

*ptrNum1 = 88
- Dereferencing operator (*) — critical distinction:
 - ptrNum1 = 10: changes the pointer itself to point to address 10 (not what you want)
 - *ptrNum1 = 88: follows the pointer to address 200 and changes the value there to 88
 - The dereference operator says: “don’t use this address; go to what this address contains”
- Why pointers work here: num1 has a longer lifetime than the display call
 - num1 is alive throughout main; display is called from within main
 - So when display executes *ptrNum1 = 88, num1 at address 200 is still valid memory
- Dangling (wild) pointer: a pointer pointing to a memory location that no longer exists
 - Caused by returning the address of a local variable after its function has returned
 - Example: display returns &data (address 600); ptr in main stores 600
 - The moment display’s closing } executes, data is destroyed; address 600 is no longer valid
 - ptr now points to a non-existent memory location: this is a dangling pointer
 - Accessing *ptr at this point yields garbage or a crash
 - Root cause: the pointed-to variable has a shorter lifetime than the pointer trying to access it
 - Contrast with the safe case: the pointed-to variable (num1) has a longer lifetime than the pointer (ptrNum1 inside display)
- Pointer to pointer (int **ptr): holds the address of another pointer
 - Allows modifying where a pointer points from within a called function
 - Example: to change ptr from pointing to num1 to pointing to num2 inside display, pass &ptr as int** xptr
 - **xptr = 5 follows two levels of indirection to reach num1’s value
 - *xptr = &data changes what ptr points to (redirects the pointer itself)
 - Not required for current coursework; introduced as general knowledge for awareness

C vs. C++: Key Distinctions

- C is a procedural language; C++ is an object-oriented extension of C
 - C++ was originally called “C with classes”; renamed to C++ using the post-increment operator metaphor
 - C++ = C + 1 (an incremented version of C)
 - Procedural programming ©:
 - Logic revolves entirely around functions (procedures)
 - Functions are global and scattered throughout the program
 - Data can be grouped using structures, but C structures hold only data, not methods
 - Data binding is possible via structures, but behavior (functions) remains separate
 - Object-oriented programming (C++):
 - Combines data and behavior (methods) into a single unit: the class
 - C++ structures can hold both data and functions, unlike C structures
 - Difference between struct and class in C++: only the default access specifier
 - struct: default access is public (maintained for backward compatibility with C)
 - class: default access is private
 - No other technical difference between them in C++
 - Additional features introduced in C++ over C:
 - Classes and objects
 - Reference variables
 - Exception handling
 - Generics (templates)
 - STL (Standard Template Library)
-

Tomorrow's Session Preview and Admin

- Topics to be covered tomorrow:
 - Reference variables in C++: what they are and how they differ from pointers
 - Call by value vs. call by reference vs. pass by value: three distinct terminologies clarified
 - Introduction to OOP: procedural vs. object-based vs. object-oriented programming
 - Features of object-oriented programming
 - Virtual functions in C++: not just what they are, but how the compiler implements them internally
 - Java introduction: why Java was created, its evolution, what you need to set it up
 - Hello World program in Java

- Do not download any Java software until the instructor demonstrates the exact download during the session
 - Specific versions and components will be specified; downloading the wrong thing causes problems
- Tomorrow's schedule (adjusted):
 - 9:30–11:30 AM: Inauguration function (formal event; attend in formals)
 - 2:00–4:00 PM: Communication session with Aditya sir
 - 4:00 PM onwards: C++ spot evaluation (5–10 minutes per student; takes 2–3 hours total)
 - Students who have already completed their spot evaluation focus on lab work
- Team structure and leads:
 - Teams of 10 already assigned; each team needs three leads: Technical Lead (TL), Communication Lead (CL), Aptitude Lead (AL)
 - TL ensures team completes technical work, joins lab on time, and does revisions and assignments
 - CL ensures English is spoken in lab hours and conducts GDs if desired
 - AL ensures aptitude modules are revised and daily coding questions are practiced (one per day, ~15 minutes)
 - Leads are expected to improve themselves as well as their team; placement of all 10 members is the goal
 - Choose leaders who are genuinely committed, not those who want the title for their resume
- Naming convention for session display names: last 3 digits of PRN + underscore + name (e.g., 094_Omkar)
- CR (Class Representative) elections coming: two TCRs and two CCRs will be selected via a democratic process including interviews
 - CRs sit above TLs/CLs/ALs in the communication hierarchy; they relay information from faculty to leads to teams
- Upcoming modules beyond this Java fundamentals course: Advanced Java (JSP, JDBC, Spring, Spring Boot, Microservices) and Web Programming Technology (React); project teams for those modules will be separate from current lab teams

Chat with meeting transcript:

<https://notes.granola.ai/t/72c62d11-a8b2-4413-855a-30029b55f052-00demib2>